TSP or the traveling salesman problem is an optimization problem. You are trying to find the shortest possible path while visiting a set of cities exactly once and returning to the starting point.

Below is the simulated annealing portion of the assignment where I had to add the acceptance rule (first set of accepts) and the cooling schedule (very bottom)

Part 5A - SA

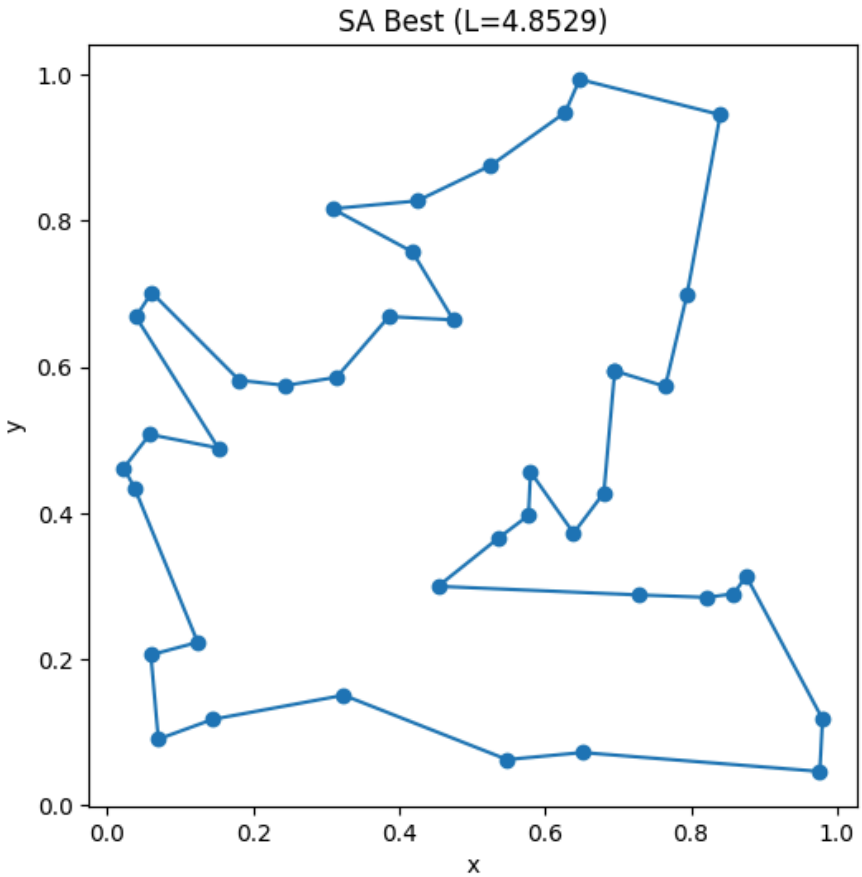
```
for it in range(cfg.iters):
    cand_tour = random_two_opt_neighbor(cur_tour, rng)
    cand_len = tour_length(cities, cand_tour)
    delta = cand_len - cur_len

    accept = False
    if delta <= 0:
        accept = True
    elif exp(-delta / T):
        accept = True


    if accept:
        cur_tour, cur_len = cand_tour, cand_len

    if cur_len < best_len:
        best_tour, best_len = cur_tour[:], cur_len

    history.append(best_len)


    T = T * SAConfig.alpha
    T = max(T, 1e-12)
```

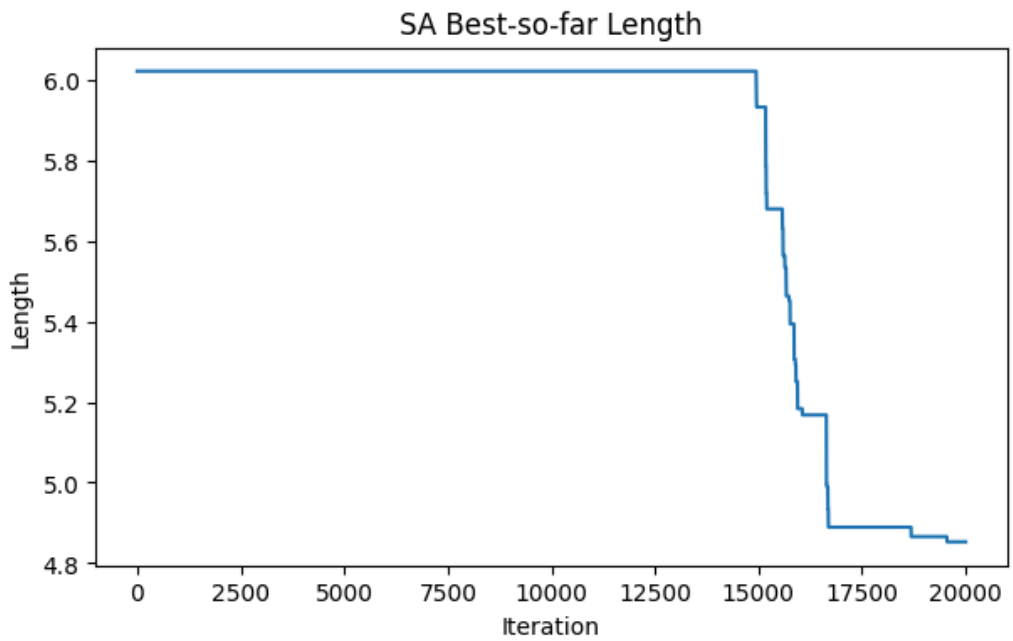
Parameters I ran with:

```
]: sa_cfg = SAConfig(iters=20_000, t0=0.2, alpha=0.9995, seed=123, report_every=2000)
   sa_best_tour, sa_best_len, sa_hist = simulated_annealing_tsp(cities, nn_tour[:], sa_cfg)

   print(f"SA best length: {sa_best_len:.4f}")
   print(f"SA improvement vs NN: {(nn_len - sa_best_len) / nn_len * 100:.2f}%")

   assert is_valid_tour(sa_best_tour, len(cities))

   plot_tour(cities, sa_best_tour, title=f"SA Best (L={sa_best_len:.4f})")
   plot_history(sa_hist, title="SA Best-so-far Length", ylabel="Length")
```

Here are the results from running it:

```
[SA] iter=  2000  T=0.07356  cur=20.3548  best=6.0214
[SA] iter=  4000  T=0.02705  cur=22.5414  best=6.0214
[SA] iter=  6000  T=0.00995  cur=21.0893  best=6.0214
[SA] iter=  8000  T=0.003659  cur=22.7149  best=6.0214
[SA] iter= 10000  T=0.001346  cur=19.9679  best=6.0214
[SA] iter= 12000  T=0.000495  cur=12.6373  best=6.0214
[SA] iter= 14000  T=0.0001821  cur=8.7450  best=6.0214
[SA] iter= 16000  T=6.696e-05  cur=5.2253  best=5.1836
[SA] iter= 18000  T=2.463e-05  cur=4.8980  best=4.8894
[SA] iter= 20000  T=9.057e-06  cur=4.8529  best=4.8529
SA best length: 4.8529
SA improvement vs NN: 19.41%
```





Part 5B GA

I had to implement the tournament_select, order_crossover_ox, and mutate_swap functions.

The tournament_select function chose random indices from pop in a loop k. It then checked if it had the lowest length, if it did the length and index was saved. Ultimately the lowest index was returned.

The order_crossover_ox function chooses two indices in parent one where the second index is greater than the first. It then cuts out the values in that range from parent one and stores them in another list at the exact same spot. It then attempts to fill in the remaining indices with values from parent two, but there can't be any duplicate values. It then returns the new list.

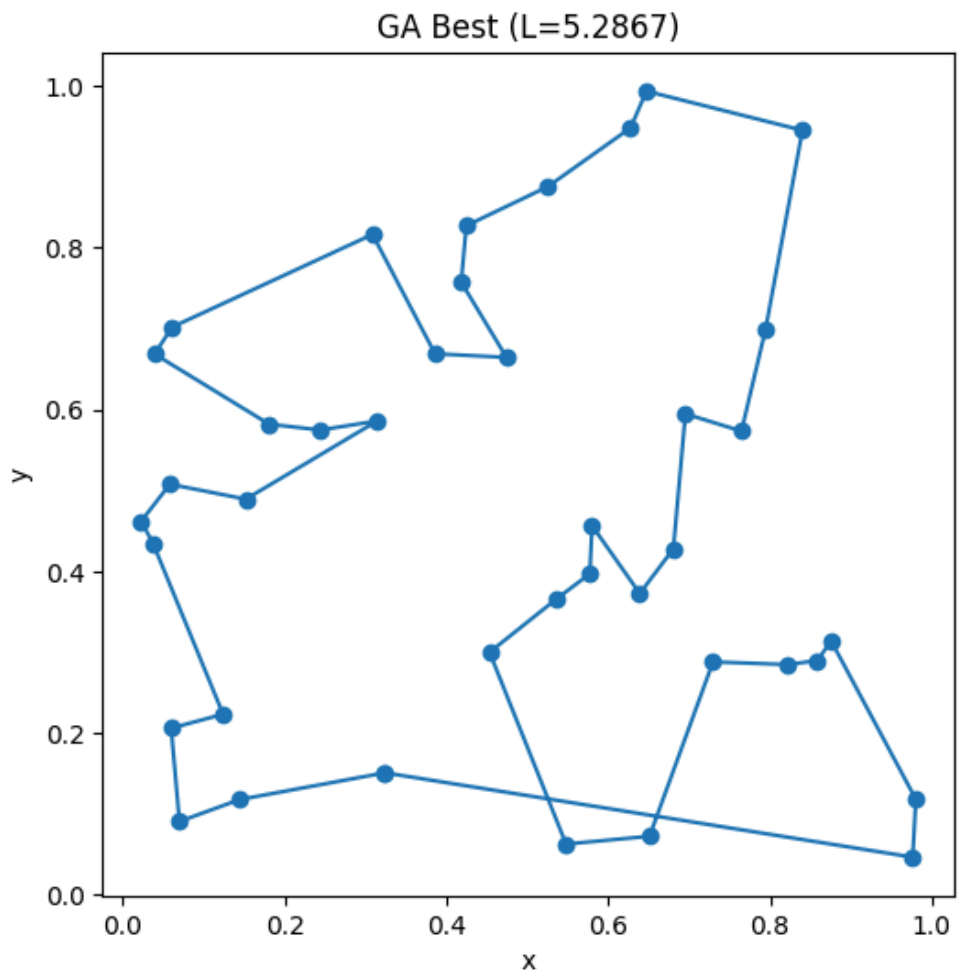
Mutate_swap is pretty straight forward as it chooses two indices from the passed in tour and swaps them.

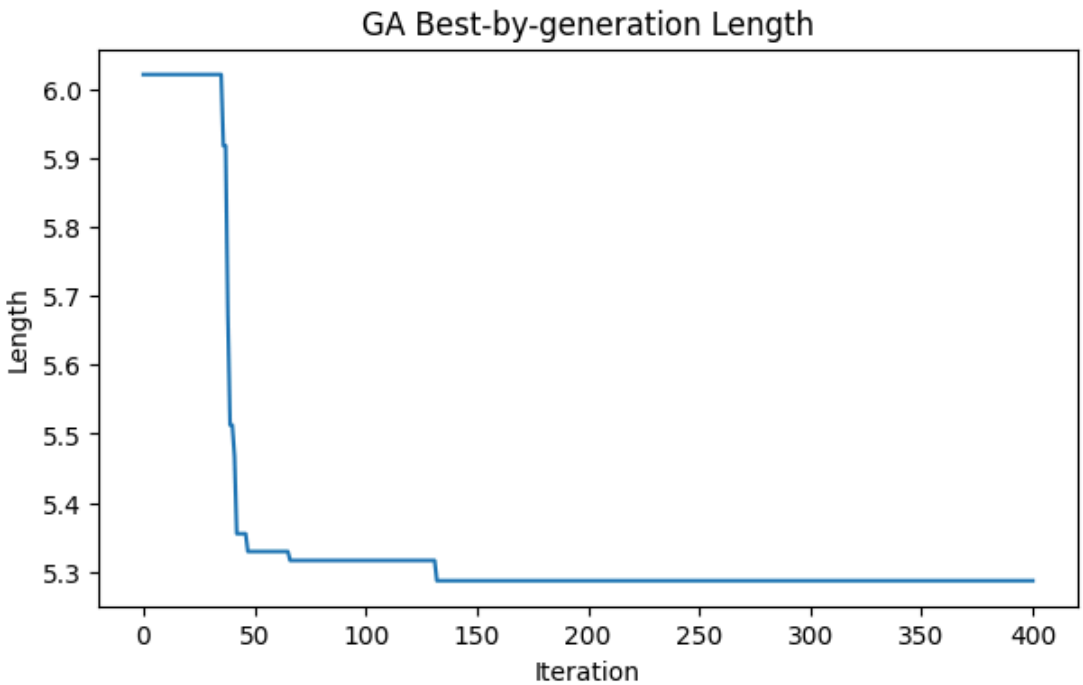
```python
def tournament_select(pop: List[Tour], lengths: List[float], k: int, rng: random.Random) -> Tour:
    lowest_idx: int = -1
    lowest_len: int = float("inf")
    for i in range(k):
        i = rng.randint(0, len(pop) - 1)
        if lengths[i] < lowest_len:
            lowest_len = lengths[i]
            lowest_idx = i
    return pop[lowest_idx]


def order_crossover_ox(parent1: Tour, parent2: Tour, rng: random.Random) -> Tour:
    length = len(parent1)

    start,end = sorted(rng.sample(range(length), 2))
    cross = [-1] * length

    cross[start:end+1] = parent1[start:end+1]
    new_parent2 = []
    for i in parent2:
        if i not in cross:
            new_parent2.append(i)
    j = 0
    for i in range(length):
        if cross[i] == -1:
            cross[i] = new_parent2[j]
            j += 1
    return cross

def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    length = len(tour)
    num1, num2 = rng.sample(range(length), 2)
    swaped_tour = tour[:]
    swaped_tour[num1], swaped_tour[num2] = swaped_tour[num2], swaped_tour[num1]
    return swaped_tour
```

Below I edited the parameters in order to get better results. I changed k to 2, mutation rate to 0.05 and elite_size to 1

```python
ga_cfg = GAConfig(pop_size=200, generations=400, tournament_k=2, crossover_rate=0.9,
                  mutation_rate=0.05, elite_size=1, seed=999, report_every=50)

ga_best_tour, ga_best_len, ga_hist = genetic_algorithm_tsp(cities, init_seed_tours=[nn_tour], cfg=ga_cfg)

print(f"GA best length: {ga_best_len:.4f}")
print(f"GA improvement vs NN: {(nn_len - ga_best_len) / nn_len * 100:.2f}%")

plot_tour(cities, ga_best_tour, title=f"GA Best (L={ga_best_len:.4f})")
plot_history(ga_hist, title="GA Best-by-generation Length", ylabel="Length")
```

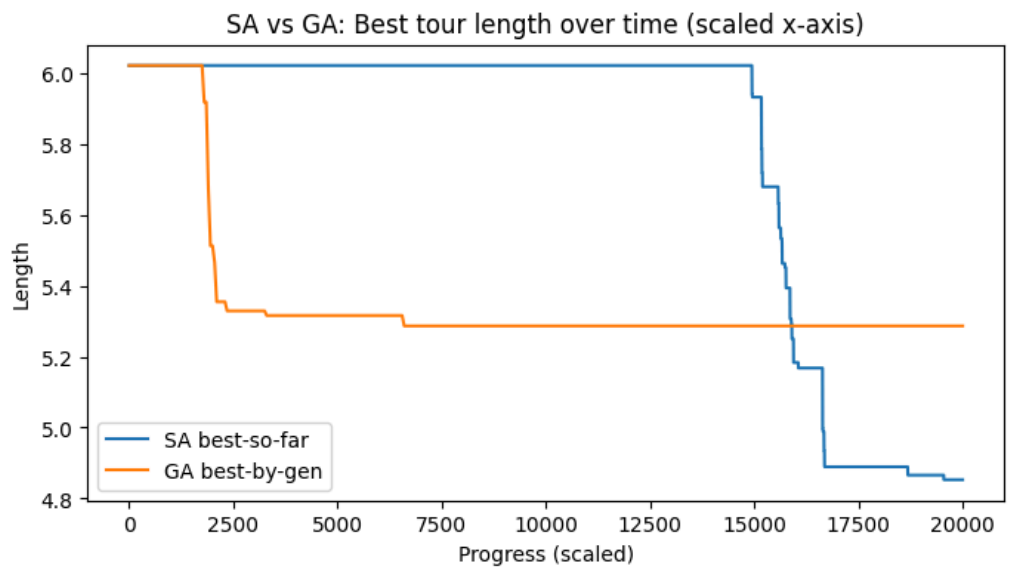
```
[GA] gen=  50  best=5.3291
[GA] gen= 100  best=5.3161
[GA] gen= 150  best=5.2867
[GA] gen= 200  best=5.2867
[GA] gen= 250  best=5.2867
[GA] gen= 300  best=5.2867
[GA] gen= 350  best=5.2867
[GA] gen= 400  best=5.2867
GA best length: 5.2867
GA improvement vs NN: 12.20%
```





Comparing the two algorithms:

```python
# After both SA and GA work, run:
plot_compare(sa_hist, ga_hist)

# And print a quick summary:
print(f"NN: {nn_len:.4f} | SA: {sa_best_len:.4f} | GA: {ga_best_len:.4f}")

grade_check(nn_len, sa_best_len, min_improvement_pct=10.0, label="SA")
grade_check(nn_len, ga_best_len, min_improvement_pct=10.0, label="GA")
```




```
NN: 6.0214 | SA: 4.8529 | GA: 5.2867
SA length: 4.8529 | Improvement vs NN: 19.41%
✅ SA passed improvement threshold!
GA length: 5.2867 | Improvement vs NN: 12.20%
✅ GA passed improvement threshold!
```

Overall the SA performed better than GA. It's improvement was 19.41% while GA's improvement was 12.20%

Part 6:

- The GA algorithm converged faster as it flattened out and stopped improving much quicker than the SA algorithm.

- SA had more variability as there were a lot of fluctuations in its best answer. GA only jumps a couple times and flattens out very quickly where SA does it much more frequently and never flattens permanently.

- Yes, SA escaped local minima. You can see in the graph that it does not stay flat for very long and it always finds a better solution even if the solution isn't much better.

- Yes GA loses diversity. In the graph it flattens out right before 150 iterations and stays flat the rest of the run. This could be due to a loss of diversity as it can't find a better solution and prematurely converges.